

Лабораторная работа №1.2

Floating a+b

Инструментарий и требования к работе

Допустимые языки	C	C++
Стандарты / версии	C23	C++23
Требования для всех работ	Правила оформления и написания работ	

Задание

Необходимо написать программу, которая позволяет выполнять арифметические действия с дробными числами в форматах с плавающей точкой по стандарту IEEE-754 и [FP Formats for Deep Learning](#). Программа должна использовать только целочисленные типы данных и вычисления.

Аргументы программе передаются через командную строку в одном из следующих вариантов:

1. `<fmt> <округление> <число>`
2. `<fmt> <округление> <операция2> <число1> <число2>`
3. `<fmt> <округление> <операция3> <число1> <число2> <число3>`

где:

- **fmt** – формат представления входных чисел.

формат	пояснение
h	числа с плавающей точкой половинной точности (half precision floating point), 16 бит.
s	формат числа с плавающей точкой одинарной точности (single precision floating point), 32 бита.
e5m2	числа с плавающей точкой низкой точности необходимо реализовать только вывод (операции для этих форматов реализовывать не требуется)
e4m3fn	
e2m1	

- операция2 – арифметическая операция: **+**, **-**, *****, **/**.
- операция3 – арифметическая операция: **mad**, **fma**. Порядок аргументов: **<число1> * <число2> + <число3>**.
- число – 1-3 числа, записанные в 16-ричной побитовой форме в любом регистре с префиксом '0x', уступающие в 32 бита. Если дано больше бит, чем нужно, то использовать младшие.
- округление – задаёт тип округления:

тип округления	пояснение	общепринятое обозначение
0	к нулю	toward_zero
1	к ближайшему чётному	toward_nearest_even
2	к $+\infty$	toward_pos_infinity
3	к $-\infty$	toward_neg_infinity

В случае присутствия операции, её результат должен быть получен в том же формате, что и входные числа в соответствии с IEEE-754. В случае получения **nan** в результате операции с не-**nan** значениями, должен получиться тихий **nan** с остальными битами мантииссы 0 и битом знака 1. Если в операции оба операнда **nan**, то выживает первый.

Результат операции или единственное входное число необходимо вывести в стандартный поток вывода в формате:

<value> <hex>

где:

- **value** – число в шестнадцатеричной показательной форме; степень в десятичном представлении; знак экспоненты выводится всегда; перед точкой всегда 1 (кроме нуля). Количество цифр после точки фиксировано и равно минимально необходимому для точного вывода

произвольного числа данного формата (соответственно, у half и single разное количество). Буквы в нижнем регистре. 0 выводится с экспонентой +0. $-\infty$ как **-inf**, $+\infty$ как **inf**. NaN выводится как **nan**.

- **hex** – представление числа аналогично входным данным. Шестнадцатеричные цифры в верхнем регистре. Количество цифр фиксировано и равно минимально необходимому для представления произвольного числа данного формата.

Примеры:

Входные аргументы	Результат
s 0 0xB9CD542	0x1.39aa84p-104 0x0B9CD542
s 0 0x8B9CD542	-0x1.39aa84p-104 0x8B9CD542
h 1 * 0x4145 0x142eB	0x1.23cp+3 0x488F
s 2 / 0x1 0x0	inf 0x7F800000
h 3 + 0xFF10 0x7F01	nan 0xFF10
s 1 mad 0x3dcccccd 0x41200000 0xbf800000	0x0.000000p+0 0x00000000
s 1 fma 0x3dcccccd 0x41200000 0xbf800000	0x1.000000p-26 0x32800000
e2m1 1 0xf	-0x1.8p+2 0xF

Проблемы

Если вы наблюдаете **✗** [ERROR] Stdout is empty, то проверьте, что вы при выводе не забываете перейти на следующую строку (например `%i\n`)